

Управление разработкой корпоративных информационных систем

XP: Принципы

XP – Обзор

- Разработано Kent Beck, Ward Cunningham, и Ron Jeffries во время их работы над проектом C3 (Chrysler Comprehensive Compensation System)
 - 1996 - Kent Beck - лидер проекта; совершенствование методологии разработки в проекте
 - 1999 - книга K.Beck “Extreme Programming Explained”
 - 2000 - Chrysler закрыл проект C3, но методология признана в области разработки ПО

XP – Ценности

- Коммуникация/взаимодействие (Communication)
- Простота (Simplicity)
- Обратная связь (Feedback)
- Смелость (Courage)
- Уважение (Respect)

Взаимодействие (Communication)

- Разработка программной системы диктует, чтобы требования к ней были **сообщены** ее разработчикам
- Практики XP – быстрые методы **создания и распространения знаний** о системе среди команды разработчиков
 - Цель – сформировать у всех разработчиков **общее видение** системы, которое совпадает с видением пользователей системы
 - XP поощряет простой дизайн, метафоричность, сотрудничество пользователей и разработчиков, интенсивное устное общение и обратную связь.

Простота (Simplicity)

- XP поощряет начать с **простого** решения и затем заниматься улучшением посредством **рефакторинга**
- Тот факт, что данная практика может повлечь большие трудозатраты завтра, для того, чтобы изменить систему, компенсируется тем, что вы **не инвестируете в возможные будущие требования**, которые могут измениться прежде, чем станут актуальными

- Кодирование и дизайн для реализации неопределенных будущих требований включает **риск растрачивания ресурсов** на нечто, что в дальнейшем может не понадобиться
- По отношению к ценности «взаимодействия» простота дизайна и кодирования должна повысить качество взаимодействия

Обратная связь (Feedback)

- **Обратная связь от системы:** разрабатывая юнит-тесты, программисты получают непосредственную обратную связь от состояния системы после реализации изменений.
- **Обратная связь от заказчика:** Функциональные тесты (или приемосдаточные тесты) пишутся заказчиком и тестерами. Они дают конкретную обратную связь по поводу состояния системы. Ревизия результатов тестирования планируется раз в 2-3 недели, так что заказчик может легко управлять разработкой.
- **Обратная связь от команды:** Когда заказчики предлагают рассмотреть новые требования во время планировочной сессии, команда сразу дает оценку времени, необходимого для реализации этих требований.
- Kent Beck: «Оптимизм – профессиональная болезнь программирования, обратная связь – лечение»

Смелость (Courage)

- Смелость позволяет разработчикам чувствовать себя комфортно во время необходимого **рефакторинга** их кода.
 - Это означает ревизию существующей системы и ее модификацию так, что будущие изменения могут быть реализованы легче. Другой пример смелости, это знать когда пора
- Выбросить код
 - Каждый программист может оказаться в тупике в конце рабочего дня при решении сложной задачи, - и найти свежее решение на следующий день, которое позволяет решить задачу за полчаса.

Уважение (Respect)

- Члены команды уважают **друг друга**, потому что они никогда не выкладывают изменения, которые не позволяют компилироваться проекту, которые приводят к тому, что существующие юнит-тесты не проходят, и, вообще **не делают того, что тормозит** работу других членов команды

- Члены команды уважают **свой труд** и поэтому постоянно борются за наивысшее качество, стремятся найти **наилучший дизайн** для своих решений путем **рефакторинга**

XP – Принципы

- Обратная связь хороша вовремя
 - Время между действием и обратной связью критично для обучения и внесения изменений
 - Контакт с заказчиком осуществляется в пределах коротких итераций
 - Юнит тесты
- Подразумеваемая простота – относиться к любой задаче так, как будто она может быть решена «предельно просто»
- Инкрементные изменения
 - например, система может иметь небольшие релизы каждые три недели
- Принятие изменений – не сопротивляться изменениям, а принимать их

XP - 12 практических методик

Процесс планирования. В XP планирование - это основная часть разработки. То, что планы могут внезапно поменяться, учитывается еще в самом начале. Аппетит заказчика приходит во время еды и нужно всегда держать ситуацию под контролем. Общение с заказчиком должно происходить как можно чаще. Это позволит точнее оценить сроки выполнения задач и внести необходимые коррективы в случае необходимости.

Быстрый выпуск версий. Как бы красиво не был описан продукт, все прелести его использования можно понять только работая с ним. Методика быстрого выпуска версий позволяет понять, чего же хочет заказчик, обеспечивая ранний выпуск рабочих версий продукта.

Метафора. Человеку намного легче запомнить отличия между двумя схожими предметами, чем знать строение предмета полностью. Методика метафора предлагает сравнивать систему (или один из ее модулей) с ее аналогами для удобства общения внутри команды и более глубокого понимания системы в целом.

Простота реализации. XP предлагает делать код программы простым. Зачем усложнять себе жизнь, если простую программу легче понимать и поддерживать и она менее подвержена ошибкам.

Опережающее тестирование. Экстремальное программирование рекомендует проверять существующий код как можно чаще. Именно поэтому и

практикуется данная методика. Тесты пишутся еще до того, как будет написан кусок кода. Это позволяет лучше понимать поставленные задачи и предоставляет возможность проверить код сразу, как только он будет написан.

Рефакторинг. Добавление новой функциональности и увеличение объема кода усложняет разработку и поиск ошибок. Решением этой проблемы является постоянная переработка (refactoring) кода. Рефакторинг - очень мощная и полезная вещь и заслуживает не то что отдельной статьи, а целых книг.

Парное программирование. Периодический просмотр чужого кода положительно влияет на его качество. Этот принцип лежит в основе парного программирования. Заключается оно в том, что два программиста одновременно работают за одним компьютером. Как ни странно, затраты на разработку не увеличиваются в два раза, а остаются приблизительно на том же уровне. С другой стороны при парном программировании повышается качество кода, осуществляется неявная и явная передача знаний, а также происходит постоянное общение между разработчиками.

Совместное владение кодом. Чаще всего при разработке программных продуктов за определенную часть кода отвечает один человек. Отпуск, увольнение или же болезнь (прости, Господи) одного из программистов может сильно затормозить разработку продукта. Именно поэтому в XP за один кусок кода отвечает не менее двух человек и любой программист может внести изменения в **любую** часть программного кода.

Продолжающаяся интеграция. Команды XP-программистов создают новые билды продукта несколько раз в день. Это позволяет всем программистам быть в курсе происходящего и предотвратить проблемы интеграции с другими продуктами и частями кода.

40-часовая неделя. Ради дела люди способны на многое, но экстремальное программирование категорически против самопожертвования разработчиков. Человек должен отдыхать. Именно тогда он достигнет максимальной производительности в рабочее время.

Контакт с заказчиком. Проблема, с которой часто сталкиваются разработчики, - это незнание предметной области, особенно, если она узкоспециализированная. Данная методика учит привлекать заказчика к процессу разработки для помощи в решении специализированных задач. Если возможности привлечь заказчика нет, иногда бывает полезным найм специалиста со стороны, работающего в необходимой отрасли.

Стандарты кодирования. Для успешной работы над проектом XP выдвигает требование применения стандартов при кодировании. Это в свою очередь обеспечивает успешность таких методик как совместное владение

кодом и парное программирование. Да и вообще, стандарты не для того, чтобы просто существовать, а для того, чтобы их придерживались.